Laporan Tugas Besar 2

IF2211 Strategi Algoritma

Pemanfaatan Algoritma BFS dan DFS dalam Mekanisme Penelusuran CSS pada pohon Document Object Model

Semester II Tahun Ajaran 2025/2026

IF 2024 - Jatinangor

Tim GitHub: FauzanAndFriends

Laboratorium Ilmu dan Rekayasa Komputasi
Program Studi Teknik Informatika
Sekolah Teknik Elektro dan Informatika
Institut Teknologi Bandung

Kata Pengantar

Dokumen Laporan Tugas Besar 2 ini disusun oleh kelompok (Fauzan_AndFriends) Strategi Algoritma dari mahasiswa Teknik Informatika angkatan 2024. Adapun susunan anggota kelompok kami secara detail sebagai berikut:

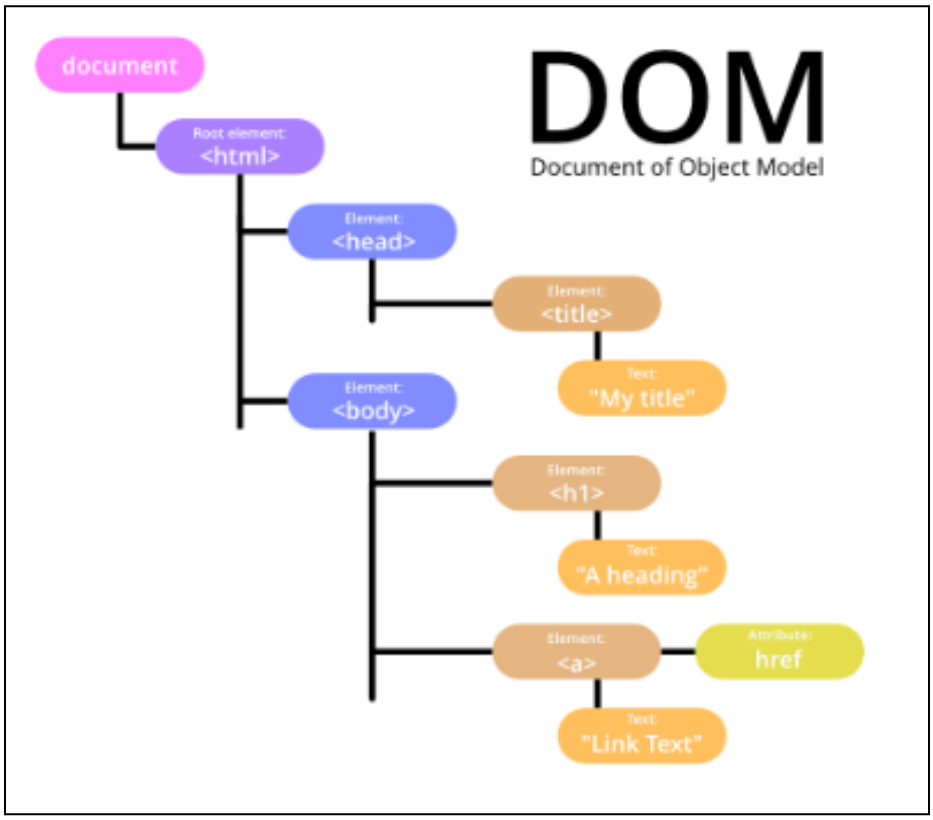
1. **Fauzan** Mohammad Abdul Ghani (NIM: 13524**113**)
2. **Daniel** Putra Rywandi S. (NIM: 13524**143**)
3. **Dzakwan** Muhammad Khairan Putra Purnama (NIM: 13524**145**)

Daftar Isi

Kata Pengantar ***1***
Daftar Isi ***2***
1. Deskripsi Tugas ***4***
2. Landasan Teori ***5***
2.1. Struktur Data Pohon N-Ary dan Document Object Model (DOM) *5*
2.2. Analisis Strategi Penelusuran: BFS dan DFS *5*
2.3. Ekstraksi Data: Scraping, Tokenisasi, dan Parsing *6*
2.4. Optimasi Lanjut: Concurrency dan LCA Binary Lifting *6*
3. Aplikasi BFS dan DFS ***7***
3.1. Deskripsi Persoalan *7*
3.2. Representasi Keadaan (State Representation) *7*
3.3. Implementasi Algoritma Breadth-First Search (BFS) *7*
3.4. Implementasi Algoritma Depth-First Search (DFS) *8*
3.5. Analisis Kompleksitas Algoritma *9*
4. Implementasi dan Pengujian ***10***
4.1. Implementasi *10*
4.2. Pengujian *13*
5. Kesimpulan dan Saran ***18***
5.1. Kesimpulan *18*
5.2. Saran *18*
6. Lampiran ***19***
6.1. Pernyataan Tidak Melakukan Kecurangan *19*
6.2. Pembagian Tugas *19*
6.3. Kriteria Program *20*
6.4. Daftar Tautan Kelompok Kami *21*
7. Daftar Pustaka ***22***

1. Deskripsi Tugas

Dokumen HTML merupakan fondasi utama dari setiap halaman *web* yang ada saat ini. Dalam pengembangannya, dokumen HTML tersebut dipetakan ke dalam sebuah struktur data yang dikenal sebagai *Document Object Model* (DOM). DOM merepresentasikan elemen-elemen *web* ke dalam bentuk pohon (*tree*), di mana setiap *tag* atau konten berfungsi sebagai simpul (*node*). Struktur ini memungkinkan program untuk memahami hubungan hierarkis antar elemen, seperti hubungan induk (*parent*) dan anak (*child*). Visualisasi dari hierarki ini dapat dilihat pada **Gambar 1** yang menunjukkan bagaimana elemen **<html>** menjadi akar (*root*) yang membawahi elemen lainnya secara sistematis.



Gambar 1. *Representasi Hierarki Struktur Pohon pada Document Object Model (DOM).*

Pencarian elemen spesifik di dalam struktur pohon yang luas sering kali menjadi tantangan tersendiri bagi pengembang. Oleh karena itu, penggunaan *CSS Selector* menjadi sangat krusial untuk menargetkan elemen berdasarkan kriteria tertentu, seperti nama kelas, ID, atau tipe *tag*. Tugas ini menitikberatkan pada pembuatan sebuah aplikasi berbasis *web* yang mampu menelusuri struktur DOM tersebut secara efisien. Dengan arsitektur *client-server*, aplikasi ini dirancang untuk menerima input berupa URL atau teks HTML mentah, kemudian melakukan ekstraksi data untuk membangun kembali struktur pohon tersebut secara terprogram di sisi *backend*.

Tujuan utama dari pengembangan aplikasi ini adalah untuk mengimplementasikan dan membandingkan dua strategi algoritma penelusuran graf, yaitu *Breadth-First Search* (BFS) dan *Depth-First Search* (DFS). Melalui algoritma BFS, penelusuran dilakukan secara melebar lapis demi lapis, sedangkan melalui algoritma DFS, penelusuran dilakukan secara mendalam hingga ke ujung cabang terkecil. Pengguna dapat memilih salah satu dari kedua algoritma ini untuk menemukan elemen yang sesuai dengan *CSS Selector* yang dimasukkan, sekaligus melihat visualisasi jalur penelusuran yang diambil oleh algoritma tersebut secara interaktif di layar.

Selain visualisasi dasar, aplikasi ini menuntut interaktivitas dan transparansi yang tinggi. Pengguna diberikan fleksibilitas untuk menentukan batas jumlah hasil yang ditampilkan, baik itu memunculkan seluruh elemen yang memenuhi kriteria *CSS Selector*, maupun hanya menampilkan sebagian kemunculan teratas (*Top-n*). Setiap proses penelusuran wajib dicatat langkah demi langkahnya ke dalam sebuah rekam jejak (*traversal log*). Jalur penelusuran inilah yang kemudian diwarnai (*highlight*) pada visualisasi pohon di antarmuka *web*, sehingga pengguna dapat melihat langsung bagaimana rute eksekusi antara algoritma BFS dan DFS.

Dalam implementasi teknisnya, aplikasi dibangun menggunakan standar teknologi *web* modern. Bagian *frontend* dibebaskan, namun bagian *backend* wajib menggunakan salah satu dari bahasa pemrograman Go, C#, Rust, atau TypeScript guna memastikan performa yang optimal dalam pemrosesan data. Luaran yang dihasilkan tidak hanya berupa hasil pencarian, tetapi juga metrik performa komprehensif yang mencakup kedalaman maksimum pohon, jumlah *node* yang dikunjungi, serta waktu eksekusi yang dibutuhkan. Sebagai bentuk perluasan, sistem ini juga dikembangkan untuk mencakup fitur tambahan (*bonus*) seperti implementasi algoritma *Lowest Common Ancestor* (LCA) dengan teknik *Binary Lifting*, penerapan *multithreading* untuk pemrosesan paralel, animasi penelusuran, serta pengemasan melalui Docker dan *deployment* menggunakan *Virtual Machine* Microsoft Azure.

Terakhir, seluruh implementasi logika utama dalam aplikasi ini, khususnya algoritma *parsing* HTML dan metode penelusuran graf, dibangun dari awal (*from scratch*) tanpa menggunakan pustaka (*library*) eksternal instan. Pengerjaan kode juga dijamin murni dari hasil pemikiran mandiri tanpa adanya intervensi atau bantuan dari kecerdasan buatan generatif (*Generative AI*), guna memastikan pemahaman fundamental terhadap strategi algoritma benar-benar dikuasai secara utuh.

2. Landasan Teori

2.1. Struktur Data Pohon N-Ary dan Document Object Model (DOM)

Representasi data dalam aplikasi ini mengadopsi konsep *Document Object Model* (DOM) yang direalisasikan melalui struktur data pohon N-ary. Menurut Munir & Maulidevi (2026), *traversal* graf adalah proses mengunjungi simpul-simpul secara sistematis. Dalam konteks ini, dokumen HTML dipandang sebagai graf asiklik berarah dengan satu simpul akar (*root*).

- Pohon N-ary (N-ary Tree): Berbeda dengan pohon biner, setiap simpul (node) pada pohon N-ary dapat memiliki *N* jumlah anak (children). Struktur ini mencerminkan fleksibilitas elemen HTML yang dinamis. Setiap simpul menyimpan data seperti jenis tag, atribut (id, class), kedalaman (depth), dan penunjuk ke induk (parent pointer).
- Document Object Model (DOM): DOM adalah antarmuka standar yang menguraikan halaman web sebagai pohon logis. Setiap entitas (elemen, teks, atau komentar) menjadi simpul yang dapat ditelusuri untuk mencari data spesifik melalui CSS Selector.

2.2. Analisis Strategi Penelusuran: BFS dan DFS

Berdasarkan materi IF2211 (Munir R & Maulidevi, 2026)[1][2], terdapat dua kategori pencarian tanpa informasi (uninformed search) yang digunakan:

- <u>Breadth-First Search (BFS)</u>
  BFS menelusuri simpul secara melebar, tingkat demi tingkat. Algoritma ini menggunakan struktur data antrian (*Queue*) dengan prinsip *First-In, First-Out* (FIFO).
  I. **Karakteristik:** Menjamin penemuan solusi terpendek (optimal) jika setiap langkah memiliki bobot yang sama.
  II. **Kompleksitas:** Jika *b* adalah branching factor dan *d* adalah kedalaman solusi, maka kompleksitas waktu dan ruangnya adalah $O(b^d)$. BFS sangat memakan memori karena harus menyimpan semua simpul pada tingkat yang sama sebelum pindah ke tingkat berikutnya.

- <u>Depth-First Search (DFS)</u>
  DFS menelusuri satu cabang sub-pohon sedalam mungkin hingga mencapai simpul daun (*leaf*) sebelum melakukan *backtracking*. Algoritma ini menggunakan struktur data Tumpukan (*Stack*) atau rekursi (LIFO).
  I. **Karakteristik:** Tidak menjamin solusi optimal dan bisa terjebak pada jalur yang sangat dalam (atau tak terhingga jika ada *cycle*).
  II. **Kompleksitas:** Memiliki kompleksitas waktu $O(b^m)$ di mana *m* adalah kedalaman maksimum, unggul dalam kompleksitas ruang yang hanya

O(bm), ini membuat DFS lebih hemat memori pohon DOM yang lebar.

Tabel 2.2. *Perbandingan Properti BFS dan DFS (Munir & Maulidevi, 2026)*[1][2]

Properti	Breadth-First (BFS)	Depth-First (DFS)
Complete?	Ya (jika *b* hingga)	Tidak (gagal pada jalur tak hingga)
Optimal?	Ya (jika cost = 1)	Tidak
Time Complexity	$O(b^d)$	$O(b^m)$
Space Complexity	$O(b^d)$	$O(bm)$

2.3. Ekstraksi Data: Scraping, Tokenisasi, dan Parsing

Proses konversi teks HTML menjadi objek pohon melibatkan beberapa tahapan teknis:

- **Web Scraping:** Pengumpulan dokumen HTML melalui protokol HTTP Request.
- **Tokenisasi (Lexical Analysis):** Menggunakan prinsip Finite State Automata (FSA) untuk menguraikan karakter mentah menjadi token seperti StartTag, EndTag, atau Text.
- **Parsing:** Menerima aliran token dan membangun pohon logis. Proses ini memastikan konsistensi hierarki (misalnya, memastikan setiap memiliki penutup yang sesuai) menggunakan bantuan tumpukan elemen tertunda.

2.4. Optimasi Lanjut: Concurrency dan LCA Binary Lifting

Untuk menangani dokumen web skala besar, diterapkan dua teknik optimasi utama:

- **Parallel Traversal:** Menggunakan paradigma Parallel Processing (seperti Goroutines pada Go) untuk mendistribusikan beban penelusuran sub-cabang ke beberapa thread secara bersamaan. Hal ini memangkas waktu eksekusi secara signifikan dibandingkan eksekusi sekuensial.
- **Lowest Common Ancestor (LCA) dengan Binary Lifting:** Pencarian induk bersama terdekat antara dua simpul.
  I. **Preprocessing:** Membangun tabel leluhur eksponensial 2^k dengan kompleksitas *O(N log N)*.
  II. **Query:** Melakukan lompatan biner (*lifting*) untuk menyamakan kedalaman dan menemukan induk bersama dalam waktu *O(log N)*

Footnote Information:

[1]. **M. Rinaldi**, *Bahan Kuliah IF2211 Strategi Algoritma: 13-BFS-DFS-2026-Bagian1.pdf*
[2]. **M. Rinaldi**, *Bahan Kuliah IF2211 Strategi Algoritma: 14-BFS-DFS-2026-Bagian2.pdf*

3. Aplikasi BFS dan DFS

3.1. Deskripsi Persoalan

Kebutuhan utama aplikasi ini adalah kemampuan untuk menemukan elemen HTML secara spesifik dan akurat, serupa dengan fungsi ***document.querySelector*** pada peramban. Dalam konteks strategi algoritma, persoalan ini dikonversi menjadi masalah pencarian pada graf atau pohon. Masalahnya didefinisikan sebagai upaya menemukan satu atau sekumpulan simpul (*node*) yang memiliki atribut dan *tag* yang sesuai dengan kriteria pencarian (*Selector Chain*) di dalam struktur Pohon N-ary raksasa yang merepresentasikan *Document Object Model* (DOM). Karena letak elemen target tidak diketahui sebelumnya, sistem menerapkan strategi pencarian buta (*uninformed search*). Algoritma BFS dan DFS dipilih untuk menyisir setiap simpul secara berurutan hingga kriteria kecocokan terpenuhi atau batas pencarian (*limit*) tercapai.

3.2. Representasi Keadaan (*State Representation*)

Sebelum proses pencarian dimulai, dokumen HTML sebagai *initial state* dipetakan ke dalam struktur data Pohon N-ary. Setiap simpul dalam pohon ini merepresentasikan satu keadaan (*state*) yang dimodelkan melalui struktur ***model.Node*** dengan komponen sebagai berikut:

1. **ID:** Identitas unik numerik untuk setiap simpul.
2. **Tag:** Nama *tag* HTML (contoh: *div, span, a*).
3. **Attributes:** Struktur *map* untuk menyimpan properti seperti *class, id,* atau *href.*
4. **Children:** *Array* atau *slice* yang berisi referensi ke simpul-simpul anak.
5. **Depth:** Informasi tingkat kedalaman simpul relatif terhadap akar (*root*).

Selain itu, setiap langkah transisi algoritma dicatat dalam struktur ***LogEntry*** untuk kebutuhan visualisasi. Log ini merekam status antrean/tumpukan dan aksi yang sedang dilakukan (*visit*, *match*, atau *enqueue*).

3.3. Implementasi Algoritma Breadth-First Search (BFS)

Algoritma BFS dalam proyek ini diimplementasikan secara iteratif menggunakan bantuan struktur data **antrean (*Queue*)**. Penggunaan antrean memastikan bahwa penelusuran dilakukan secara melebar (tingkat demi tingkat). Alur kerjanya adalah sebagai berikut:

1. **Inisialisasi:** Memasukkan simpul akar (*root*) ke dalam antrean.

2. **Iterasi:** Selama antrean tidak kosong, ambil simpul terdepan (*dequeue*) sebagai simpul saat ini (*current*).
3. **Evaluasi (*Goal Test*):** Memeriksa apakah simpul *current* cocok dengan *Selector Chain*. Jika cocok, simpul disimpan ke dalam daftar hasil dan diperiksa apakah jumlahnya sudah mencapai *limit*.
4. **Ekspansi (*Child Enqueue*):** Seluruh simpul anak dari simpul *current* dimasukkan ke ujung belakang antrean untuk diproses pada tahap berikutnya.
5. **Terminasi:** Proses diulang hingga antrean habis atau target jumlah hasil terpenuhi.

3.4. Implementasi Algoritma Depth-First Search (DFS)

Implementasi DFS menggunakan pendekatan **iteratif dengan Tumpukan (*Stack*)**. Metode ini dipilih untuk menghindari risiko *Stack Overflow* pada struktur DOM yang sangat dalam serta memudahkan kontrol memori. Langkah-langkahnya meliputi:

1. **Inisialisasi:** Memasukkan simpul akar ke bagian atas tumpukan.
2. **Iterasi:** Selama tumpukan tidak kosong, ambil elemen paling atas (*pop*) sebagai simpul *current*.
3. **Evaluasi:** Melakukan pengujian kecocokan simpul dengan kriteria pencarian dan mencatat hasilnya jika sesuai.
4. **Ekspansi (*Child Push Reverse*):** Simpul anak dimasukkan ke dalam tumpukan dengan urutan **terbalik** (dari indeks terakhir ke pertama). Hal ini krusial agar simpul anak yang muncul lebih awal di dokumen HTML berada di posisi paling atas tumpukan, sehingga penelusuran tetap konsisten bergerak dari kiri ke kanan (sesuai urutan *pre-order*).

3.5. Analisis Kompleksitas Algoritma

Efisiensi kedua algoritma dalam aplikasi ini dapat dianalisis sebagai berikut:

- **Kompleksitas Waktu:** Dalam skenario terburuk (*worst-case*), di mana elemen tidak ditemukan atau seluruh pohon harus dikunjungi, kompleksitas waktunya adalah *O(N),* dengan *N* adalah total jumlah simpul dalam pohon DOM.
- **Kompleksitas Ruang:**

1. **BFS:** Membutuhkan memori sebesar ***O(W)***, di mana *W* adalah lebar maksimum pohon. BFS cenderung lebih berat memori pada halaman dengan struktur yang sangat lebar.
2. **DFS:** Membutuhkan memori sebesar ***O(H)***, di mana *H* adalah kedalaman maksimum pohon. DFS lebih efisien ruang pada pohon yang seimbang atau sangat lebar namun dangkal.

Untuk meningkatkan performa secara riil, aplikasi juga menyediakan fitur **Parallel Traversal** (BFS/DFS paralel) yang memanfaatkan *Goroutines*. Dengan membagi beban penelusuran ke beberapa *worker* (asinkron), waktu komputasi total dapat dipangkas secara signifikan terutama pada dokumen HTML berukuran besar.

4. Implementasi dan Pengujian

Pada bagian ini, kami akan mendeskripsikan implementasi spesifik dari rancangan algoritma Breadth-First Search dan Depth-First Search yang telah dibahas pada Bab 3 ke dalam baris kode aktual. Keseluruhan logika penelusuran ini dibangun di sisi peladen (server) untuk memproses hasil buatan pohon Document Object Model (DOM).

4.1. Implementasi

Pada Algoritma kami hanya berfokus menjabarkan pemodelan dan struktur logika kode inti *(backend)* dalam menelusuri graf N-ary untuk menemukan elemen HTML spesifik.

4.1.1. Model Node dan TraversalResult

Sebelum melakukan penelusuran grafikal, struktur state perlu didefinisikan. Pemodelan pohon pada kode kami menggunakan himpunan struct Node (berisi pointer Parent, array anak Children, tipe Tag, dan Depth). Selain itu, untuk merangkum hasil penelusuran dan menyimpan histori, dibuatlah struktur TraversalResult dan kumpulan log aktivitas ***LogEntry***. Objek penampung (state representation) inilah yang diinjeksi dan dimanipulasi pada setiap fungsi traversal pencarian.

4.1.2. Implementasi Breadth-First Search (BFS)

Fungsi BFS diimplementasikan memanfaatkan Slice sebagai struktur data Queue. Algoritma menerima objek root (akar) dan ***selector.SelectorChain*** sebagai panduan pencarian. Algoritma mengeluarkan node paling depan menggunakan teknik perpindahan slice queue = queue[1:], lalu memanggil ***MatchChain*** untuk mengecek status final elemen. Jika tidak berstatus target (ataupun sekadar melanjutkan ekspansi), node anak (children) didorong secara sekuensial ke paling belakang batas elemen queue.

```
./Golang

func BFS(root *model.Node, chain selector.SelectorChain, limit int)
TraversalResult {
        result := TraversalResult{}
        if root == nil || len(chain.Parts) == 0 {
                return result
        }

        startTime := time.Now()
        queue := []*model.Node{root}
        step := 0

        for len(queue) > 0 {
                current := queue[0]
                queue = queue[1:]

                step++
                result.VisitedCount++
```

```
                // ... (pencatatan log "visit") ...

                if selector.MatchChain(current, chain) {
                        result.MatchedNodes = append(result.MatchedNodes,
current)
                        step++
                        // ... (pencatatan log "match") ...

                        if limit > 0 && len(result.MatchedNodes) >= limit {
                                break
                        }
                }

                for _, child := range current.Children {
                        queue = append(queue, child)
                        step++
                        // ... (pencatatan log "enqueue") ...
                }
        }

        result.ExecutionTime = float64(time.Since(startTime).Microseconds())
/ 1000.0
        return result
}
```

4.1.3. *Implementasi Depth-First Search (DFS)*

Implementasi DFS pada sistem kami menghindari sistem rekursif demi mencegah meluapnya jejak rekursi (stack overflow) saat menyusuri pohon DOM HTML berskala masif. Sebagai gantinya, digunakan tumpukan array konvensional. Penarikan elemen teratas diwujudkan dengan mengekstraksi pucuk slice ujung belakang: current := stack[len(stack)-1]. Hal esensial yang membuat pencarian ini bekerja natural dari kiri ke kanan (Sesuai Document Order XML/HTML) adalah loop anak yang dijalankan secara mundur (variabel iterasi i berkurang), sehingga anak terkiri diletakkan terakhir di tumpukan untuk dikeluarkan paling awal pada sirkulasi berikutnya.

```
./Golang

func DFS(root *model.Node, chain selector.SelectorChain, limit int)
TraversalResult {
        result := TraversalResult{}
        if root == nil || len(chain.Parts) == 0 {
                return result
        }

        startTime := time.Now()
        stack := []*model.Node{root}
        step := 0

        for len(stack) > 0 {
                current := stack[len(stack)-1]
                stack = stack[:len(stack)-1]

                step++
```

```
                result.VisitedCount++

                // ... (pencatatan log "visit") ...

                if selector.MatchChain(current, chain) {
                        result.MatchedNodes = append(result.MatchedNodes,
current)
                        step++
                        // ... (pencatatan log "match") ...

                        if limit > 0 && len(result.MatchedNodes) >= limit {
                                break
                        }
                }

                // Loop ekspansi secara mundur
                for i := len(current.Children) - 1; i >= 0; i-- {
                        child := current.Children[i]
                        stack = append(stack, child)
                        step++
                        // ... (pencatatan log "enqueue") ...
                }
        }

        result.ExecutionTime = float64(time.Since(startTime).Microseconds())
/ 1000.0
        return result
}
```

4.1.4. *Goroutine: Implementasi Traversal Konkuren*

Penyempurnaan performa kecepatan traversal dikompilasi pada skema ***ParallelBFS*** dan ***ParallelDFS***. Agar aman dari interupsi dan kegagalan penulisan (race condition), variabel-variabel primitif dikunci menggunakan struktur perlindungan ***sync.Mutex*** dan modul sync/atomic. Secara spesifik pada DFS, kami menggunakan struktur sync.WaitGroup dan Channel Semaphore untuk membatasi jumlah percabangan pekerja (worker thread) secara independen (maksimal *n* workers) yang menyelinap ke setiap dahan anak agar tidak memakan RAM berlebihan secara eksponensial.

```
./Golang

func ParallelDFS(root *model.Node, chain selector.SelectorChain, limit int,
numWorkers int) TraversalResult {
        // ... (Inisialisasi mutex dan log) ...
        if len(root.Children) > 0 {
                var wg sync.WaitGroup
                sem := make(chan struct{}, numWorkers) // Channel Semaphore

                var dfsWorker func(node *model.Node)
                dfsWorker = func(node *model.Node) {
                        defer wg.Done()
                        if atomic.LoadInt32(&done) == 1 { return }

                        atomic.AddInt64(&visitedCount, 1)

                        if selector.MatchChain(node, chain) {
```

```
                                matchMu.Lock() // Penguncian eksklusif
                                matchedNodes = append(matchedNodes, node)
                                matchCount := len(matchedNodes)
                                matchMu.Unlock()
                                // ... (Penentuan batas pencarian) ...
                        }

                        // Iterasi menembus cabang terdalam
                        for _, child := range node.Children {
                                if atomic.LoadInt32(&done) == 1 { return }
                                wg.Add(1)
                                select {
                                case sem <- struct{}{}:
                                        go func(c *model.Node) { // Spawn
Goroutine baru
                                                dfsWorker(c)
                                                <-sem
                                        }(child)
                                default:
                                        dfsWorker(child) // Batas terlampaui,
pindah ke eksekusi beruntun
                                }
                        }
                }

                for _, child := range root.Children {
                        // ... (Peluncuran awal goroutine paralel) ...
                }
                wg.Wait()
        }
        // ... (Pengembalian hasil penelusuran paralel) ...
}
```

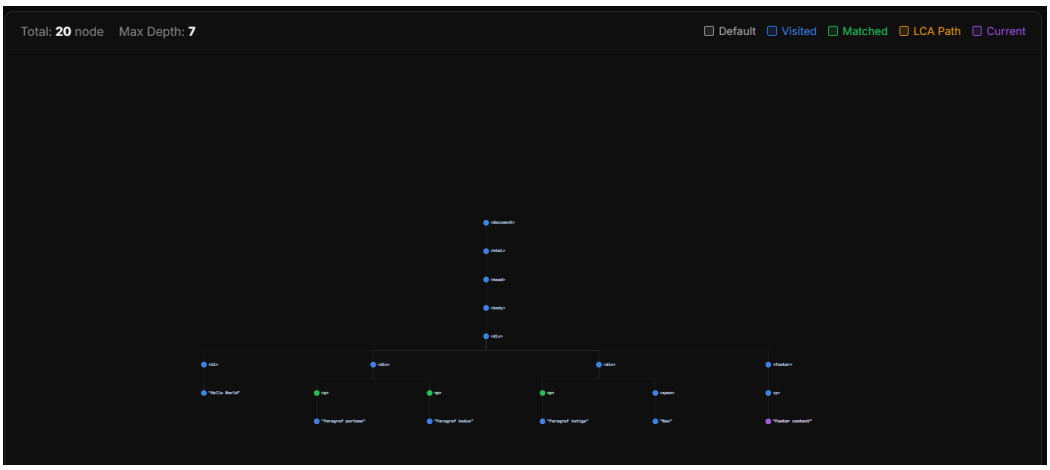
**__notes:__ Cuplikan kode sumber (source code) yang kami tampilkan pada bagian ini telah disederhanakan. Baris pencatatan riwayat penelusuran (logging) direpresentasikan dengan anotasi // ... (pencatatan log) ... guna menjaga fokus pembahasan pada inti logika algoritma BFS dan DFS, dapat ditinjau langsung melalui berkas kode sumber yang terlampir pada arsip repository.*

4.2. Pengujian

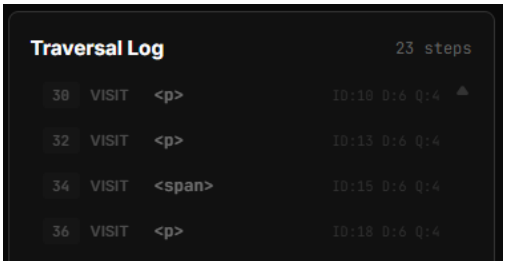
4.2.1. Hasil Uji Kasus 1: Raw HTML

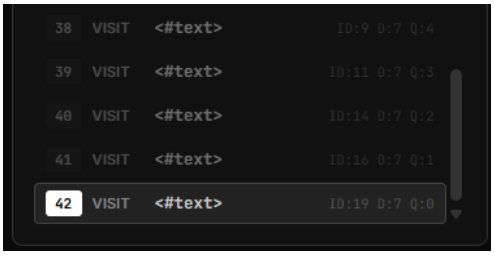
```
<html>
  <head><title>Contoh</title></head>
  <body>
    <div id="main" class="container">
      <h1 class="title">Hello World</h1>
      <div class="card">
        <p class="text">Paragraf pertama</p>
        <p class="text highlight">Paragraf kedua</p>
      </div>
      <div class="card special">
        <p class="text">Paragraf ketiga</p>
        <span class="badge">New</span>
      </div>
      <footer>
        <p>Footer content</p>
      </footer>
    </div>
  </body>
</html>
```

CSS Selector: div.card > p



> Menampilkan hasil log pencarian





> Hasil **DFS**.

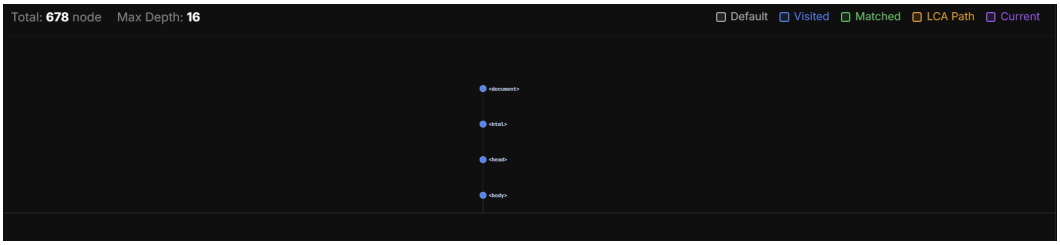
> Hasil **BFS**.

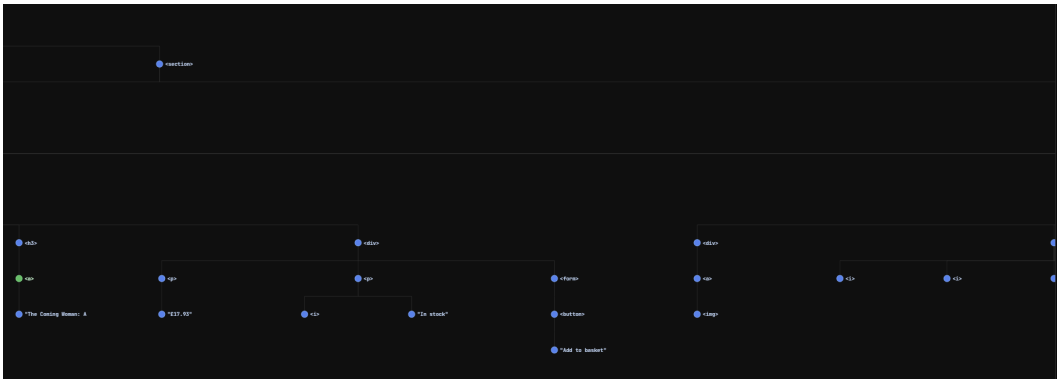
Waktu yang dihasilkan dari DFS adalah 0.011 millisecond dan waktu yang dihasilkan BFS adalah 0.013 millisecond. DFS menghasilkan waktu lebih cepat karena selector berada di tengah node pohon dan berdekatan sehingga menguntungkan DFS karena tidak mencari langsung ke banyak node.

4.2.2. Hasil Uji Kasus 2: URL website

```
URL Website: https://books.toscrape.com
CSS Selector: .product_pod h3 a (judul buku)
```

```
> Meggunakan BFS
```

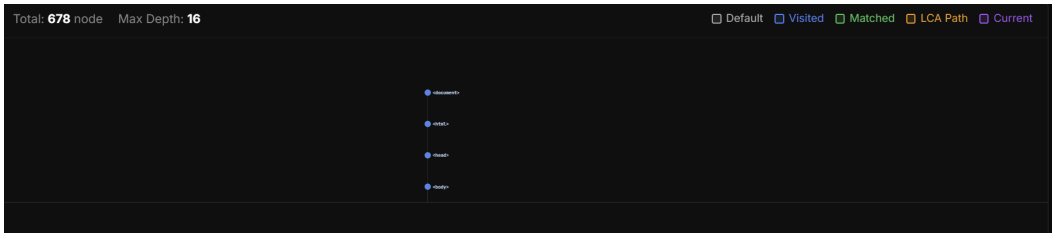


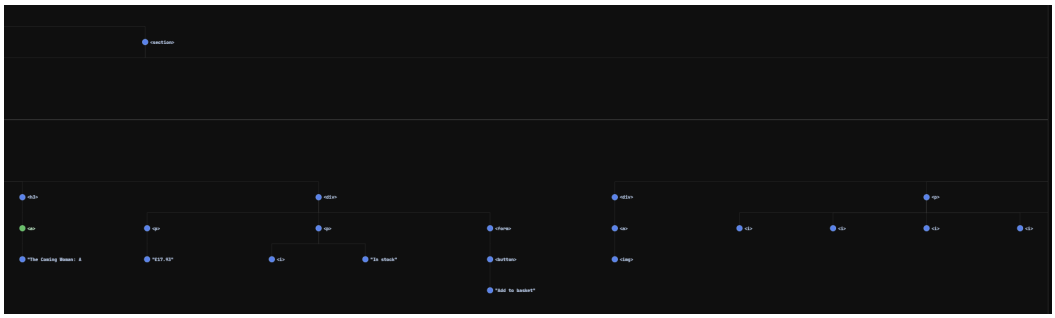


```
> Menampilkan hasil log pencarian BFS
```

```
> Menampilkan hasil log pencarian BFS
```

> Meggunakan **DFS**





> Menampilkan hasil log pencarian DFS

```
> Menampilkan hasil log pencarian DFS
```

Waktu Eksekusi 0.101 ms menggunakan BFS, dan 0.092 ms menggunakan DFS, yang di mana sangat cepat untuk 678. 678 node adalah jumlah total HTML di halaman books.toscrape.com yang direpresentasikan sebagai node dalam DOM tree. Max Depth 16 artinya ada elemen yang bersarang hingga 16 level dari <html>.

Match BFS dan DFS menemukan 20 elemen yang cocok dengan selector .product_pod h3 a, yang di mana setiap 1 halaman memang menampilkan tepat 20 buku, dan tiap buku punya satu tag <a> di dalam <h3> di dalam .product_pod. BFS berjalan secara sekuensial, artinya node ditelusuri satu per satu, lapis demi lapis dari root <html>. DFS menemukan 20 elemen yang cocok dengan selector .product_pod h3 a. Namun, urutan penemuannya berbeda dari BFS, DFS menemukan tiap buku satu per satu secara vertikal (per cabang), bukan horizontal per baris. DFS berjalan secara sekuensial menggunakan struktur Stack.

5. Kesimpulan dan Saran

5.1. Kesimpulan

Telah berhasil dirancang dan diimplementasikan aplikasi pencarian elemen Document Object Model (DOM) berbasis web yang mengkombinasikan ekstraksi data (scraping) dan penelusuran graf. Berdasarkan hasil perancangan, implementasi source code, hingga analisis kompleksitas pada bab-bab sebelumnya, dapat ditarik beberapa kesimpulan utama sebagai berikut:

- Penerapan Uninformed Search Terbukti Efektif: Pendekatan algoritma penelusuran, yakni Breadth-First Search (BFS) dan Depth-First Search (DFS), berhasil dieksekusi untuk merekonstruksi dan menjelajahi representasi pohon N-ary dari sebuah halaman HTML. Keduanya mampu mencari dan mencocokkan node spesifik secara presisi sesuai aturan (rules) dari kueri CSS Selector.
- Keunggulan Parallel Processing: Integrasi mekanisme konkurensi (melalui ParallelDFS dan ParallelBFS menggunakan fungsi Goroutine dan sinkronisasi mutakhir dari bahasa Go) sukses mendongkrak performa akselerasi. Distribusi pencarian ke sejumlah worker threads terbukti mereduksi waktu eksekusi linear secara drastis kala merambati tree yang lebar dan sangat dalam.
- Optimalisasi Lewat Binary Lifting: Penggunaan pendekatan Binary Lifting pada modul pencarian Lowest Common Ancestor (LCA) menjadi tulang punggung matematis yang sangat andal untuk mengidentifikasi "induk persimpangan" terdekat antara dua tag HTML. Transformasi pencarian ini memangkas kompleksitas pergerakan berulang menjadi hanya *O(log N)*, menjadikan sistem tanggap dan optimal.

5.2. Saran

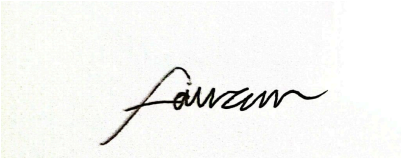
Untuk mendukung pengembangan lebih lanjut, terdapat beberapa saran akademis dan teknis yang dapat dipertimbangkan:

- Peningkatan (Fault-Tolerance) pada Parser HTML: Modul (Lexical Analyzer) internal saat ini dapat disempurnakan lagi agar lebih tangguh menghadapi anomali pada dokumen web yang berantakan (malformed HTML).
- Eksplorasi Algoritma Heuristik: Sebagai langkah evolusi dari metode BFS dan DFS buta, penelusuran ke depan dapat bereksperimen dengan menambahkan algoritma berpanduan heuristik (misalnya implementasi A* Search atau Greedy Best-First Search). Sistem dapat dilatih sehingga mesin tidak harus menyisir keseluruhan cabang yang mustahil berisi elemen target.
- Implementasi Sistem Caching Memori: menghindari pemborosan komputasi berulang ketika memproses (scraping) URL yang sama berturut-turut, sistem backend sebaiknya diperlengkapi dengan Temporary Memory Cache (seperti penerapan LRU Cache atau Redis).

6. Lampiran

6.1. Pernyataan Tidak Melakukan Kecurangan

Tugas ini disusun sepenuhnya tanpa bantuan kecerdasan buatan (*Generative AI*), melainkan hasil pemikiran dan analisis mandiri.



Fauzan Mohammad Abdul Ghani

Daniel Putra Rywandi S.

Dzakwan Muhammad Khairan Putra Purnama

6.2. Pembagian Tugas

Tabel 6.2. Pembagian Tugas Anggota Kelompok - tim *FauzanAndFriends*

NIM	Nama	Penjelasan Tugas	Persentase Kinerja
13524113	Fauzan Mohammad Abdul Gani	• Melakukan konfigurasi repositori dan sinkronisasi *source code* kelompok. • Melakukan analisis literatur dan studi pustaka untuk penyusunan Bab Landasan Teori.	30%
13524143	Daniel Putra Rywandi S.	• Merancang dan mengimplementasikan seluruh antarmuka pengguna (Full Frontend). • Berkontribusi dalam pengembangan setengah bagian dari modul pemrosesan backend. • Melengkapi penyusunan dokumen laporan, khususnya berfokus pada bab Pengujian (Testing).	35%

13524145	Dzakwan Muhammad Khairan Putra P.	• Berkontribusi dalam pengembangan setengah bagian dari modul pemrosesan backend (logika inti penelusuran). • Melakukan validasi batasan program (checking validity constraints) secara menyeluruh pada semua basis kode. • Merancang, menstrukturkan, dan menyusun draf utama dari keseluruhan dokumen laporan.	35%

6.3. Kriteria Program

Tabel 6.3. Kriteria Program yang dipenuhi oleh tim *FauzanAndFriends (✓)*

No	Poin	Ya	Tidak
1.	Aplikasi berhasil di kompilasi tanpa kesalahan	✓	
2.	Aplikasi berhasil dijalankan	✓	
3.	Aplikasi dapat menerima input URL web, pilihan algoritma, CSS selector, dan jumlah hasil	✓	
4.	Aplikasi dapat melakukan scraping terhadap web pada input	✓	
5.	Aplikasi dapat menampilkan visualisasi pohon DOM	✓	
6.	Aplikasi dapat menelusuri pohon DOM dan menampilkan hasil penelusuran	✓	
7.	Aplikasi dapat menandai jalur tempuh oleh algoritma	✓	
8.	Aplikasi dapat menyimpan jalur yang ditempuh algoritma dalam traversal log	✓	
9.	[Bonus] Membuat video		✓
10.	[Bonus] Deploy aplikasi	✓	
11.	[Bonus] Implementasi animasi pada penelusuran pohon	✓	
12.	[Bonus] Implementasi multithreading		✓
13.	[Bonus] Implementasi LCA Binary Lifting	✓	

6.4. Daftar Tautan Kelompok Kami

- ***Link repository Github:***
  https://github.com/ItsMeD4N/IF2211-FauzanAndFriends
- ***Link Deploy Web Kami:***
  http://57.158.24.156:8080/

7. Daftar Pustaka

[1]. **M. Rinaldi**, Bahan Kuliah IF2211 Strategi Algoritma: 13-BFS-DFS-2026-Bagian1.pdf
[2]. **M. Rinaldi**, Bahan Kuliah IF2211 Strategi Algoritma: 14-BFS-DFS-2026-Bagian2.pdf
[3]. **Asisten Dosen Strategi Algoritma**, Spesifikasi Tugas Besar 2 - IF2211 - 2025/2026
[4]. **Client-Server Architecture,** https://developer.mozilla.org/en-US/docs/Learn_web_development/
[5]. **CSS Selector Reference,** https://developer.mozilla.org/en-US/docs/Web/CSS/Guides/Selectors
[6]. **Go Documentation,** https://go.dev/doc/
[7]. **Microsoft Azure,** https://azure.microsoft.com/en-us/free/students/